CS524-Introduction to Cloud Computing – Spring 2026

Final Project – Analysis Report

AI-Powered Secure Web Application on AWS

Aisiri Mandya Rajashekar

arajashe@stevens.edu

CWID: 20043750

Stevens Institute of Technology

Live URL: *https://d169r8mwjk1yr9.cloudfront.net*

1. Introduction

For this project I built a web application that analyzes text using AI, hosted entirely on AWS. The idea was to take a block of text the user types in, send it to a backend running on AWS Lambda, and get back things like whether the text sounds positive or negative, what named entities are in it, and how readable it is. The frontend is a static website I designed myself and hosted on S3, delivered through CloudFront over HTTPS so everything is secure.

I completed this entirely inside the AWS Academy Lab environment using the US East (N. Virginia) region. The project pushed me to actually connect multiple AWS services together rather than just using them individually, which was a good learning experience overall.

2. Architecture and Data Flow

The architecture I designed is fully serverless, meaning there are no EC2 instances or traditional servers involved anywhere. I chose this approach because it keeps costs low, removes the overhead of managing servers, and scales automatically.

Here is how a typical request flows through the system:

- The user opens the website by visiting the CloudFront URL in their browser.
- CloudFront pulls the static files (HTML, CSS, JavaScript) from my S3 bucket and delivers them over HTTPS.
- When the user types some text and clicks Analyze, the browser sends a POST request directly to the Lambda Function URL.
- Lambda runs my NLP analysis on the text — sentiment, entities, key phrases, and readability — and saves the result as a JSON file in the S3 results/ folder.
- The results are returned to the browser and displayed on the page.
- CloudWatch automatically captures logs and metrics from Lambda throughout this whole process.

The four main AWS services in this project and what each one does:

Service	What it does in this project
Amazon S3	Stores the website files and saves the analysis results as JSON
Amazon CloudFront	Delivers the website over HTTPS globally using a CDN
AWS Lambda	Runs the Python NLP backend when a request comes in, no server needed
Amazon CloudWatch	Logs every Lambda invocation and monitors errors and duration

3. What I Did in Each Phase

Phase 1 – Architecture Diagram and Cost Estimate

I started by drawing the architecture diagram showing how all the services connect. Then I went to the AWS Pricing Calculator and added S3, Lambda, CloudFront, and CloudWatch with minimal usage values. Getting the total to $0.00 took a bit of tweaking — CloudWatch kept showing a small charge because I had accidentally selected SNS mobile notifications which are not free. Once I set those to zero and reduced the CloudFront data transfer to 0.001 GB, everything came down to $0.00.

Phase 2 – S3 Bucket and Static Website

I created an S3 bucket called cs-524-aisiri-website in us-east-1. I had to uncheck Block all public access and acknowledge the warning. After that I enabled static website hosting with index.html as the index document, added the public read bucket policy, and turned on SSE-S3 encryption under Default encryption. I saved the bucket website endpoint URL for later.

Phase 3 – Lambda Function

I created a Lambda function named cs524-ai-analyzer using Python 3.12 and the LabRole execution role. The first thing I did after creating it was increase the timeout to 30 seconds since the default 3 seconds would not be enough for the NLP processing. I pasted the code provided by the course assistant, updated the BUCKET_NAME variable to match my bucket, and deployed it. Testing with the provided sample input returned statusCode 200 with full analysis results. I then created a Function URL with Auth type NONE and CORS enabled for all origins.

Phase 4 – Uploading the Website and Connecting to Lambda

I built my own frontend with a modern minimalist design — a text input area, an Analyze button, and separate result sections for sentiment, entities, key phrases, and readability. Before uploading I updated the API_URL in script.js with my Lambda Function URL. After uploading all three files to S3 I ran a few test analyses and confirmed the results folder in S3 was being populated with JSON files after each request.

Phase 5 – CloudFront

I created a CloudFront distribution using the Pay-as-you-go plan, selected my S3 bucket as the origin, and clicked Use website endpoint when the yellow banner appeared. I set the default root object to index.html and waited about 10 minutes for the distribution status to change from Deploying to Enabled. Once it was live, I opened the CloudFront HTTPS URL and the analyze function worked immediately — this also fixed the connection issue I had been seeing when testing via the S3 HTTP endpoint.

Phase 6 – CloudWatch Monitoring

I opened CloudWatch and navigated to Log groups where I found /aws/lambda/cs524-ai-analyzer with log streams from my test runs. I created a dashboard called CS524-AI-Dashboard and added widgets for Invocations, Errors, and Duration. I then set up an alarm on the Errors metric with a threshold of 5 over a 5-minute period, created an SNS topic, and confirmed the subscription from my email.

4. Security

I tried to apply security at every layer of the application rather than just in one place.

For data in transit, all traffic goes through CloudFront which enforces HTTPS. Any HTTP requests get redirected to HTTPS automatically. The Lambda Function URL is also HTTPS only, so the full request path from browser to backend is encrypted.

For data at rest, I enabled SSE-S3 encryption on the bucket. Every file stored there — including the analysis JSON results — is encrypted using S3-managed keys without any extra setup needed.

For access control, the Lambda function runs under the LabRole which has only the permissions it needs. The bucket policy only allows s3:GetObject which means external users can read the website files but cannot write to or delete from the bucket.

One thing I would change in a real deployment is the Lambda Function URL auth type. I used NONE for this project because it was required for the frontend to call it without authentication, but in production I would add some form of API key or IAM-based authorization to prevent abuse.

5. How the AI/NLP Works

The Lambda function does not use any external NLP library or AWS service like Comprehend. Everything is implemented in plain Python using regex and word lists. There are four analysis components:

Sentiment Analysis

The function has two sets of words — one for positive words like great, love, and excellent, and one for negative words like terrible, fail, and broken. It tokenizes the input text, counts how many words fall into each category, and computes percentage scores. If the positive score is above 0.03 and higher than the negative score, the sentiment is POSITIVE. If they are close together, it returns MIXED. This is a simple but effective approach for the kinds of short texts the app is designed for.

Named Entity Recognition

Entities are found using regex patterns for things like email addresses, URLs, phone numbers, dates, and multi-word proper nouns. There is also a hardcoded list of known locations like Hoboken, New Jersey, and New York that gets checked against the text directly. Each entity is returned with its type, where it appeared in the text, and a confidence score.

Key Phrase Extraction

The function splits the text into sentences and then builds 2 to 4 word sequences, filtering out any phrases made up entirely of common stop words. Phrases are scored based on length — longer phrases score higher since they tend to carry more meaning. The top 10 are returned.

Readability

Readability is calculated using the Flesch Reading Ease formula. It looks at the average number of words per sentence and the average number of syllables per word to produce a score between 0 and 100. A higher score means easier to read. The score is also mapped to a plain English label like Easy or Fairly Difficult. Word count, sentence count, and average words per sentence are returned alongside the score.

6. Challenges I Ran Into

S3 Bucket Permission Error

When I first tried to create the bucket I got an error saying s3:CreateBucket permission is required. I thought there was something wrong with my configuration but it turned out my AWS Academy lab session had just expired. The fix was simple — I went back to Canvas, ended the lab, and restarted it. After the green indicator came back on I was able to create the bucket without any issues.

Failed to Fetch Error on the Website

This one took longer to figure out. After uploading my files and opening the website through the S3 endpoint, clicking Analyze kept giving me a 'Failed to fetch' error. I checked the Lambda URL in script.js and it was correct. I tested the Lambda directly in the console and it worked fine. I tried re-uploading the files and doing hard refreshes but nothing helped.

Eventually I realized the issue was that the S3 website endpoint uses HTTP while the Lambda Function URL uses HTTPS. Browsers treat this as mixed content and block the request by default. The real fix was completing Phase 5 — once I had the CloudFront distribution set up and accessed the site via HTTPS,

everything worked straight away. This made me appreciate why CloudFront is part of the required architecture and not just optional.

7. Cost Analysis

I used the AWS Pricing Calculator to estimate what this deployment would cost. With minimal usage values the total came to $0.00 per month across all four services. The table below shows the free tier limits that cover this project:

Service	Free Tier Allowance	Monthly Cost
S3	5 GB storage, 20K GET, 2K PUT requests/month	$0.00
Lambda	1M requests, 400K GB-seconds/month	$0.00
CloudFront	1 TB data transfer, 10M requests/month	$0.00
CloudWatch	10 custom metrics, 5 GB logs/month	$0.00
Total	**All services within free tier**	**$0.00**

The cost stayed at zero because Lambda only charges when it actually runs, S3 only charges for what you store, and CloudFront gives a very large free tier that is more than enough for a project like this. Serverless was clearly the right choice here from a cost perspective.

8. Lessons Learned

The biggest thing I took away from this project was how much easier it is to build something when you do not have to manage servers. With Lambda I just wrote the Python code, uploaded it, and it ran. No installation, no configuration, no worrying about whether the server is up. That is a genuinely different way of thinking about building software and I can see why companies are moving towards it.

The mixed content issue taught me something I will not forget — when you build a web app, every single part of it needs to use the same protocol. It sounds obvious in hindsight but I did not realize the browser would completely block a request just because the page was on HTTP and the API was on HTTPS. CloudFront was the right fix because it made the whole thing HTTPS end to end.

I also learned to always check whether my lab session is still active before blaming my configuration. When the bucket creation failed I spent some time checking my settings when the real issue was just that my session had expired. That is something specific to the Academy environment but good to know.

Setting up CloudWatch at the end showed me how important monitoring is even for a small project. Seeing the invocation count go up each time I ran a test, and knowing there is an alarm ready to fire if something goes wrong, made the whole deployment feel much more production-like than just having a website that works.

9. Conclusion

Overall this project came together well. The application is live at https://d169r8mwjk1yr9.cloudfront.net and works as expected — you type in some text, hit Analyze, and within a second or two you get back the sentiment, any named entities, the key phrases, and a readability score. All of that is happening in a Lambda function running in the cloud with no server for me to manage.

Going through all six phases gave me a solid end-to-end understanding of how a real cloud application is put together. It is not just about writing code — it is about hosting it correctly, securing it, connecting the

pieces, and then actually being able to see what it is doing once it is deployed. That combination is what made this project useful beyond just the technical implementation.